

ПРАВИТЕЛЬСТВО РОССИЙСКОЙ ФЕДЕРАЦИИ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»
Факультет компьютерных наук
Образовательная программа «Программная инженерия»

УДК 004.023

ОТЧЕТ

по исследовательскому курсовому проекту

на тему

**«Оптимизация алгоритмов маршрутизации на примере задачи
нескольких коммивояжеров»**

по направлению подготовки бакалавров 09.03.04 «Программная инженерия»

Выполнил
студент группы БПИ246
Д. Д. Купцов

Научный руководитель
Ермаков Андрей Максимович

Москва 2026

РЕФЕРАТ

Отчет содержит описание исследовательского курсового проекта, посвященного задаче множественного коммивояжера и практическому сравнению эвристических алгоритмов маршрутизации. В работе рассматривается постановка mTSP с одним депо и целевой функцией MINSUM, при которой требуется минимизировать суммарную длину всех маршрутов. Объектом исследования являются процессы маршрутизации и распределения точек между несколькими агентами, предметом исследования являются эвристические и метаэвристические методы решения mTSP при ограниченном времени вычислений.

Цель работы --- разработать и экспериментально оценить масштабируемые эвристические методы оптимизации маршрутов для задачи множественного коммивояжера. Для достижения цели были изучены классические подходы к TSP и mTSP, реализованы базовые алгоритмы и семейство LKH-подобных оберток, подготовлен экспериментальный контур на C++ и Python, проведены сравнения с OR-Tools, преобразованием TSP $\rightarrow$ mTSP и внешним решателем LKH-3. Полученные результаты показывают, что LKH-подобные методы существенно превосходят простые конструктивные эвристики по качеству маршрутов, однако при сравнении с полноформатным LKH-3 возникает компромисс: собственные быстрые версии работают заметно быстрее, но обычно уступают по значению целевой функции.

Ключевые слова: задача коммивояжера, множественный коммивояжер, mTSP, маршрутизация, эвристические алгоритмы, Lin--Kernighan--Helsgaun, локальный поиск, MINSUM, вычислительный эксперимент.

Нужно добавить перед сдачей. После финальной правки нужно обновить строку с объемом работы: количество страниц, таблиц, рисунков, источников и приложений. Это лучше сделать после окончательной компиляции PDF.

Содержание

ОПРЕДЕЛЕНИЯ, ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ	5
ВВЕДЕНИЕ	7
1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ	10
1.1 Классическая задача коммивояжера	10
1.2 Задача множественного коммивояжера	10
1.3 Эвристические и метаэвристические подходы	11
1.4 Масштабирование и candidate sets	12
2 ПОДРОБНЫЙ РАЗБОР ЭВРИСТИК MTSP	13
2.1 mtsp_rand_nn_solver: случайное разбиение и nearest neighbor	13
2.2 mtsp_greedy_2opt_solver: жадное построение и 2-opt . .	14
2.3 mtsp_grasp_solver: GRASP, RCL и межмаршрутные обмены	15
2.4 LKH и LKH-3 как внешний ориентир качества	16
2.5 Собственные LKH-подобные версии lkh-wrapper	18
2.6 FILO2 и крупномасштабные VRP-эвристики	19
2.7 ITSNA и двухэтапные mTSP-эвристики из литературы	20
2.8 Кластерно-генетические подходы и MMCTSP	21
2.9 POPMUSIC и построение candidate sets	22
2.10 Сравнительная характеристика эвристик	22
3 ПОСТАНОВКА ЗАДАЧИ И РЕАЛИЗАЦИЯ	24
3.1 Формальная постановка в проекте	24
3.2 Экспериментальный контур	24
3.3 Набор реализованных алгоритмов	25
3.4 Особенности LKH-подобной реализации	26
4 МЕТОДИКА ВЫЧИСЛИТЕЛЬНОГО ЭКСПЕРИМЕНТА	27
4.1 Семейства тестовых инстансов	27
4.2 Метрики	27
4.3 Валидация результатов	28
5 РЕЗУЛЬТАТЫ И ИХ АНАЛИЗ	28

5.1	Базовое сравнение на равномерных инстансах	28
5.2	Сравнение с OR-Tools	29
5.3	Расширенный benchmark по семействам инстансов	30
5.4	Эволюция LKH-подобных версий	31
5.5	Крупные инстансы и проблема масштабирования	31
5.6	Сравнение v12 с внешним LKH-3 baseline	32
5.7	Интерпретация результатов	33
6	ДОСТОВЕРНОСТЬ, ОГРАНИЧЕНИЯ И РИСКИ	34
	ЗАКЛЮЧЕНИЕ	34
	СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	36
	ПРИЛОЖЕНИЕ А. ЧТО НУЖНО ДОБАВИТЬ В ФИНАЛЬНУЮ ВЕРСИЮ ПЕРЕД СДАЧЕЙ	38
	ПРИЛОЖЕНИЕ Б. РЕКОМЕНДУЕМЫЙ СОСТАВ ПОЛНОЙ ТАБ- ЛИЦЫ ЭКСПЕРИМЕНТОВ	39

ОПРЕДЕЛЕНИЯ, ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ

Термин	Определение
TSP	Travelling Salesman Problem, задача коммивояжера: требуется построить кратчайший замкнутый маршрут, проходящий через все заданные вершины ровно один раз.
mTSP	Multiple Travelling Salesman Problem, задача множественного коммивояжера: требуется построить несколько маршрутов, начинающихся и заканчивающихся в общем депо, так чтобы каждая клиентская вершина была посещена ровно одним маршрутом.
Депо	Специальная вершина, из которой начинаются и в которой завершаются маршруты всех агентов.
Агент, коммивояжер	Исполнитель, которому назначается отдельный маршрут в решении mTSP.
MINSUM	Целевая функция mTSP, равная суммарной длине всех маршрутов. В данной работе меньшая MINSUM означает более качественное решение.
MINMAX	Альтернативная целевая функция, в которой минимизируется длина самого длинного маршрута. В работе она используется как вспомогательный контекст, но не как основной критерий.
2-opt	Локальное улучшение маршрута, при котором удаляются два ребра и фрагмент маршрута разворачивается, если это уменьшает длину.
k-opt	Обобщение 2-opt: из маршрута удаляется k ребер, после чего фрагменты соединяются новым способом.
LKH	Lin--Kernighan--Helsgaun, семейство высокоэффективных эвристик для TSP, основанных на переменной глубине локального поиска и кандидатных ребрах.

Термин	Определение
Candidate set	Ограниченный набор перспективных соседей вершины, используемый для ускорения локального поиска и отказа от полного перебора всех пар вершин.
GRASP	Greedy Randomized Adaptive Search Procedure, жадно-рандомизированная процедура построения и улучшения решений.
OR-Tools	Библиотека Google для решения задач оптимизации, в работе используется как внешний эвристический ориентир для маршрутизации.
FILLO2	Эвристический подход для крупномасштабных задач маршрутизации, используемый как внешний ориентир по идеям масштабирования.
Benchmark	Набор тестовых экземпляров задачи, на котором сравниваются алгоритмы.
Relative gap	Относительное отклонение решения от выбранного ориентира: $100 \cdot (A - B)/B$, где A --- значение исследуемого алгоритма, B --- значение базового решения.

ВВЕДЕНИЕ

Задачи маршрутизации относятся к числу наиболее практически значимых задач комбинаторной оптимизации. Они возникают в логистике, транспортном планировании, управлении цепями поставок, распределении сервисных бригад, проектировании производственных процессов и других областях, где требуется упорядочить перемещение между множеством точек. Классическим математическим ядром таких задач является задача коммивояжера, в которой необходимо найти маршрут минимальной длины, проходящий через все вершины ровно один раз и возвращающийся в исходную вершину [1,2].

Несмотря на простую формулировку, TSP относится к NP-трудным задачам. Это означает, что при росте числа вершин полный перебор решений становится неприемлемым уже на сравнительно малых размерах. В прикладных постановках ситуация обычно еще сложнее: маршрут строится не для одного исполнителя, а для нескольких агентов, которые стартуют из общего депо и должны совместно обслужить набор клиентов. Такая постановка называется задачей множественного коммивояжера, или mTSP. В отличие от TSP, здесь требуется не только определить порядок обхода вершин, но и распределить вершины между маршрутами [4].

В данной работе рассматривается вариант mTSP с одним депо и целевой функцией MINSUM. Для практических задач такая постановка естественна: если несколько машин или исполнителей обслуживают один и тот же набор точек, то суммарная длина маршрутов может быть связана с расходом топлива, временем работы, стоимостью обслуживания или суммарной нагрузкой на систему. При этом получение точного оптимума для задач средней и большой размерности обычно не является реалистичной целью. Более важным становится поиск достаточно качественного приближенного решения в заранее заданный вычислительный бюджет.

Актуальность исследования определяется двумя обстоятельствами. Во-первых, mTSP является базовой моделью для более сложных задач маршрутизации, включая варианты Vehicle Routing Problem. Во-вторых, современные эвристические методы показывают, что сильное качество решения часто достигается не за счет полного перебора, а за счет аккуратного ограничения области поиска: кандидатных ребер, локальных перестановок, кластеризации, разреженных соседств и процедур повторного улучшения [3,7,8,10]. Поэтому практический

интерес представляет не только выбор известного решателя, но и анализ того, какие инженерные решения делают алгоритм масштабируемым.

Объектом исследования являются процессы маршрутизации и распределения маршрутов между несколькими агентами в задачах комбинаторной оптимизации. Предметом исследования являются эвристические и метаэвристические алгоритмы решения mTSP, а также их влияние на качество маршрутов и время вычислений при росте размерности задачи.

Цель работы --- разработать и проанализировать эффективные эвристические методы оптимизации маршрутов для задачи множественного коммивояжера. Для достижения цели в работе решаются следующие задачи:

1. изучить постановку TSP и mTSP, основные целевые функции и ограничения;
2. проанализировать существующие эвристические подходы, включая локальный поиск, GRASP, LKH и методы построения candidate sets;
3. реализовать единый экспериментальный контур для запуска алгоритмов и проверки корректности решений;
4. подготовить несколько семейств синтетических mTSP-инстансов, отражающих разные геометрические сценарии;
5. сравнить базовые эвристики, собственные LKH-подобные версии и внешние ориентиры по качеству, времени и валидности решений;
6. сформулировать выводы о применимости разработанных подходов и ограничениях текущей реализации.

В качестве рабочей гипотезы принимается следующее положение: эвристики, использующие ограниченные множества кандидатных ребер, локальный поиск и LKH-подобные процедуры улучшения, позволяют получать более качественные решения mTSP по MINSUM, чем простые конструктивные алгоритмы, при сопоставимых или приемлемых вычислительных затратах. Дополнительно проверяется более осторожная инженерная гипотеза: быстрые собственные версии могут быть полезны при жестком временном бюджете, даже если по качеству они уступают полноформатному внешнему LKH-3.

Методологическая основа работы включает анализ научной литературы, математическую формализацию задачи, программную реализацию алгоритмов, вычислительный эксперимент, валидацию маршрутов и сравнительный анализ результатов. Основные эксперименты проводятся на синтетических

инстансах с разными распределениями точек: равномерным, кластерным, кластерным со смещенным депо, смешанным с выбросами и стрессовыми сценариями с большим числом агентов.

Нужно добавить перед сдачей. Перед сдачей желательно добавить точную конфигурацию компьютера: процессор, объем RAM, ОС, компилятор, режим сборки Release/Debug и наличие OpenMP. В исходных материалах есть результаты запусков, но нет полного описания аппаратной среды, а для экспериментальной главы это важная деталь.

1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

1.1 Классическая задача коммивояжера

Задача коммивояжера формулируется следующим образом. Пусть задано множество вершин $V = \{0, 1, \dots, n - 1\}$ и функция расстояния $d(i, j)$ между каждой парой вершин. Требуется найти перестановку вершин, задающую замкнутый маршрут минимальной длины:

$$\min_{\pi} \left(d(\pi_n, \pi_1) + \sum_{i=1}^{n-1} d(\pi_i, \pi_{i+1}) \right). \quad (1)$$

В евклидовой постановке вершины задаются координатами на плоскости, а расстояние вычисляется как длина отрезка между точками. Такая постановка удобна для экспериментов, поскольку позволяет генерировать тестовые данные контролируемым образом и сохраняет связь с прикладными задачами маршрутизации.

Точные методы решения TSP имеют большую теоретическую ценность, но для крупных экземпляров используются ограниченно. В ранних работах были предложены процедуры ветвей и границ, динамического программирования и другие точные подходы, однако рост пространства решений делает их дорогостоящими [1]. Поэтому в практических задачах широко применяются эвристики. Уже классический алгоритм Lin--Kernighan показал, что локальный поиск переменной глубины способен давать оптимальные или близкие к оптимальным решения на широком классе тестов [1]. Позднее Helsgaun развил этот подход в LKH, усилив его использованием кандидатных ребер и более сложной стратегии поиска [2].

1.2 Задача множественного коммивояжера

mTSP является естественным обобщением TSP. Пусть задано m агентов и общее депо 0. Решение представляет собой набор маршрутов:

$$R_s = (0, v_1^s, v_2^s, \dots, v_{k_s}^s, 0), \quad s = 1, \dots, m, \quad (2)$$

где каждая вершина из $V \setminus \{0\}$ встречается ровно в одном маршруте. В варианте MINSUM целевая функция имеет вид:

$$F_{\text{sum}}(R_1, \dots, R_m) = \sum_{s=1}^m \sum_{i=0}^{k_s} d(v_i^s, v_{i+1}^s), \quad v_0^s = v_{k_s+1}^s = 0. \quad (3)$$

В литературе также часто рассматривается MINMAX, где минимизируется длина самого длинного маршрута. Эти цели отражают разные практические смыслы. MINSUM ориентирован на суммарную стоимость обслуживания, а MINMAX --- на балансировку нагрузки между агентами. В данной работе основной критерий --- MINSUM, но баланс маршрутов дополнительно анализируется через отношение длины самого длинного маршрута к средней длине маршрута.

Обзор Bektaş показывает, что mTSP имеет несколько эквивалентных и близких формулировок, а также может решаться через преобразования к TSP, целочисленное программирование и эвристические методы [4]. Для задач большой размерности особенно важны эвристики, поскольку даже если точный метод может решить малый пример, он не дает приемлемого времени на крупных данных.

1.3 Эвристические и метаэвристические подходы

Простейший класс методов составляют конструктивные эвристики. Они строят решение последовательно, выбирая следующую вершину по локальному правилу, например по ближайшему соседу. Такие методы очень быстры, но часто принимают ранние решения, которые затем трудно исправить. В проекте к этой группе относится алгоритм rand+nn: он распределяет вершины между маршрутами с элементом случайности и ближайшего соседа.

Второй класс методов основан на локальном поиске. Для каждого маршрута можно применять 2-opt: если замена двух ребер уменьшает длину маршрута, соответствующий фрагмент разворачивается. В проекте этот подход представлен алгоритмом 2opt+greed. Он сильнее чистого ближайшего соседа, но все еще ограничен, потому что улучшает уже построенные маршруты и не всегда хорошо перераспределяет вершины между агентами.

Третий класс составляют рандомизированные многостартовые процедуры. GRASP строит несколько решений с использованием ограниченного спис-

ка лучших локальных кандидатов, затем улучшает каждое решение и выбирает лучшее [9]. В реализации проекта `grasp` использует `randomized restricted candidate list`, внутримаршрутный 2-opt и межмаршрутные обмены. Такой подход заметно повышает качество, но его время растет с числом итераций и размером инстанса.

Отдельное место занимают LKH-подобные методы. Их сила состоит в том, что поиск не рассматривает все возможные ребра, а концентрируется на малом наборе перспективных соседей. Для TSP эта идея поддержана классическими работами Lin--Kernighan и Helsgaun [1,2]. Для mTSP LKH-3 использует преобразование задач маршрутизации к стандартному симметричному TSP и штрафные функции для ограничений [3]. В частности, для mTSP с одним депо возможно добавить копии депо и получить структуру, распадающуюся на несколько маршрутов.

1.4 Масштабирование и candidate sets

Ключевая инженерная проблема крупных TSP/mTSP-инстансов --- невозможность рассматривать полный граф. Если для каждой вершины проверять все остальные вершины, построение соседств требует порядка $O(n^2)$ сравнений. Для $n = 100000$ это уже непрактично. Поэтому современные эвристики строят разреженный граф кандидатов.

В работах Taillard и Helsgaun по POPMUSIC показано, что качественные candidate sets можно строить с эмпирической сложностью ниже квадратичной даже для очень крупных TSP-инстансов [7]. Похожую идею используют крупномасштабные VRP-решатели: они ограничивают локальный поиск гранулярными соседствами, динамическими областями изменений и кэшами затронутых вершин [8]. Для данной работы это важно не как прямое заимствование готового алгоритма, а как подтверждение общего принципа: масштабируемость достигается через сужение пространства ходов и отказ от полного перебора.

В собственных версиях `lkh-wrapper` этот принцип реализован через геометрические candidate sets, пространственную сетку, кэширование расстояний, ограниченный 2-opt и time-budget. В более поздних версиях добавлены кластерные начальные маршруты, процедуры перераспределения блоков между маршрутами и отдельные бюджеты на построение первого решения и последующее улучшение.

2 ПОДРОБНЫЙ РАЗБОР ЭВРИСТИК MTSP

В данной главе отдельно рассматриваются эвристики, которые используются в проекте или служат для него методологическими ориентирами. Такое разделение важно по двум причинам. Во-первых, итоговое качество решения mTSP определяется не только формальной постановкой, но и тем, как именно построено начальное распределение вершин, какие локальные ходы разрешены и насколько сильно ограничено пространство поиска. Во-вторых, разные алгоритмы полезны в разных режимах: один метод удобен как быстрый baseline, другой дает лучшее качество на малых инстансах, третий показывает, какие инженерные идеи нужны для масштабирования до десятков и сотен тысяч вершин.

Основная целевая функция в проекте --- MINSUM. Поэтому при сравнении эвристик главный вопрос формулируется так: насколько эффективно алгоритм уменьшает суммарную длину всех маршрутов при сохранении валидности решения. При этом отдельно учитывается баланс маршрутов, поскольку слишком длинный отдельный маршрут может быть нежелателен даже тогда, когда суммарная длина выглядит приемлемой.

Нужно добавить перед сдачей. После выбора рисунков из папки docs/materials в начало этой главы стоит добавить обзорную схему: ``конструктивные эвристики → локальный поиск → многостартовые методы → LKH/LKH-3 → крупномасштабные эвристики FILO2/POPMUSIC". Такой рисунок поможет показать, как связаны простые решатели проекта и методы из статей.

2.1 `mtsp_rand_nn_solver`: случайное разбиение и nearest neighbor

Решатель `mtsp_rand_nn_solver` зарегистрирован в проекте под именем `rand+nn`. Это самый простой внутренний baseline, который нужен не столько для получения сильного результата, сколько для проверки экспериментального контура и оценки нижнего уровня качества простых конструктивных решений.

Алгоритм работает в два этапа. Сначала все клиентские вершины, то есть вершины $1, \dots, n - 1$, перемешиваются с использованием фиксируемого seed. Затем перемешанный список распределяется между коммивояжерами по кругу. После этого для каждой полученной группы строится маршрут ближайшего

соседа: текущая вершина последовательно соединяется с ближайшей еще не посещенной вершиной из той же группы.

Сильная сторона этого подхода --- очень малое время работы и воспроизводимость при заданном seed. Он удобен как технический ориентир: если более сложная эвристика проигрывает rand+nn, значит в ней, скорее всего, есть ошибка реализации или неудачная настройка параметров. Кроме того, случайное начальное разбиение позволяет быстро получать несколько разных решений без изменения основной логики.

Главный недостаток метода состоит в том, что распределение клиентов между маршрутами никак не связано с геометрией задачи. Две близкие точки могут оказаться у разных агентов, а один маршрут может получить группу, которая геометрически плохо собирается в короткий цикл. Локальный nearest neighbor внутри группы не исправляет ошибку назначения, потому что не переносит вершины между маршрутами. Поэтому rand+nn ожидаемо дает худшее качество среди реализованных внутренних методов, но остается полезным как базовая точка сравнения.

2.2 `mtsp_greedy_2opt_solver`: жадное построение и 2-opt

Решатель `mtsp_greedy_2opt_solver` зарегистрирован под именем `2opt+greed`. Он усиливает простую конструктивную схему за счет более осмысленного выбора следующей вершины и последующего локального улучшения каждого маршрута.

Построение начинается с m пустых маршрутов, каждый из которых стартует в депо. Далее алгоритм циклически проходит по коммивояжерам. Для текущего коммивояжера выбирается ближайшая еще не посещенная вершина относительно конца его текущего маршрута. Выбранная вершина добавляется в маршрут, помечается как посещенная, и процесс продолжается, пока все клиентские вершины не будут распределены. После закрытия маршрутов возвратом в депо для каждого маршрута отдельно запускается процедура 2-opt.

2-opt улучшает порядок обхода внутри одного маршрута. Если две дуги маршрута можно заменить двумя другими дугами так, что цикл станет короче, соответствующий фрагмент маршрута разворачивается. В контексте данного решателя это означает, что жадное распределение вершин остается прежним, но порядок посещения внутри каждого маршрута становится более аккурат-

ным. Поэтому 2opt+greed обычно заметно лучше rand+nn: он устраняет часть пересечений и явно неудачных участков внутри маршрутов.

Ограничение метода связано с тем, что он почти не пересматривает ранние решения. Если вершина была назначена одному агенту, 2-opt не передаст ее другому агенту, даже если такой перенос уменьшил бы MINSUM. Кроме того, жадный выбор ближайшей вершины может создавать локально хорошие, но глобально неудачные продолжения. По этой причине 2opt+greed является сильнее простого baseline, но все еще уступает многостартовым и LKH-подобным методам.

Жадное назначение ближайшей вершины каждому агенту → закрытие маршрутов в депо → внутримаршрутный 2-opt → проверка валидности и расчет MINSUM

Рис. 1: Логика эвристики 2opt+greed

Нужно добавить перед сдачей. На место рисунка выше можно поставить собственную визуализацию: слева маршруты после жадного построения, справа те же маршруты после 2-opt. Лучше выбрать небольшой инстанс на 30--50 вершин, чтобы были видны пересечения, которые убирает локальный поиск.

2.3 mtsp_grasp_solver: GRASP, RCL и межмаршрутные обмены

mtsp_grasp_solver реализует GRASP-подход и зарегистрирован как grasp. В отличие от 2opt+greed, он не строит единственное детерминированное решение. Алгоритм многократно создает кандидатные решения с элементом случайности, улучшает их и сохраняет лучшее найденное значение MINSUM.

На этапе построения для каждого агента формируется список доступных городов, отсортированный по расстоянию от текущей вершины маршрута. Вместо того чтобы всегда брать ближайший город, алгоритм выбирает случайную вершину из restricted candidate list --- списка из нескольких лучших локальных кандидатов. Размер этого списка задается параметром rcl. Если rcl=1, поведение становится близким к жадному выбору; при большем rcl поиск получает больше разнообразия, но может чаще делать слабые локальные шаги.

После построения каждого маршрута применяется 2-opt. Затем запускается межмаршрутная фаза: алгоритм перебирает пары маршрутов и пробует поме-

нять местами внутренние вершины. Если обмен уменьшает MINSUM, он принимается, а затронутые маршруты дополнительно улучшаются 2-opt. Этот шаг принципиально отличает grasp от чистого 2opt+greed: теперь алгоритм может частично исправлять неудачное распределение вершин между агентами.

Преимущество GRASP состоит в сочетании трех механизмов: рандомизации, локального улучшения и выбора лучшего решения из нескольких запусков. Такой подход устойчивее к локальным ошибкам начального построения и обычно дает более качественные маршруты, чем чистые конструктивные методы. Недостаток --- рост времени работы. На каждом старте требуется сортировать локальных кандидатов, выполнять 2-opt и проверять межмаршрутные обмены. При увеличении числа итераций качество может улучшаться, но вычислительная стоимость растет почти прямо вместе с числом попыток.

Параметр	Смысл	Влияние на результат
iters	число независимых построений решения	повышает шанс найти более короткий MINSUM, но линейно увеличивает время работы
rcl	размер restricted candidate list	управляет балансом между жадностью и разнообразием поиска
seed	начальное состояние генератора случайных чисел	делает стохастический запуск воспроизводимым

Нужно добавить перед сдачей. Для этой части хорошо подойдет рисунок из статьи про GRASP или собственная схема RCL: полный список кандидатов, отсортированный по расстоянию, и выделенный верхний фрагмент, из которого случайно выбирается следующая вершина. Если готового рисунка в материалах нет, его можно сделать самостоятельно в стиле схемы алгоритма.

2.4 LKH и LKH-3 как внешний ориентир качества

Lin--Kernighan и LKH относятся к более сильному классу локального поиска переменной глубины. В обычном 2-opt фиксируется простой тип хода:

заменить две дуги. В Lin--Kernighan логика гибче: алгоритм строит последовательность замен ребер и продолжает ее, пока есть надежда получить выигрыш. Helsgaun в LKH усилил этот подход *candidate sets*, специальными правилами выбора ребер и настройками, которые позволяют не просматривать полный граф [1,2].

Для mTSP особенно важен LKH-3. Он расширяет LKH на ряд ограниченных задач маршрутизации, включая варианты TSP, VRP и mTSP. Общая идея состоит в сведении исходной задачи к стандартному симметричному TSP с дополнительными структурами и штрафными функциями [3]. Для задачи нескольких коммивояжеров это позволяет описывать число агентов, депо и целевую функцию, например MINSUM. В практическом смысле LKH-3 выступает сильным внешним *baseline*: если собственная эвристика приближается к нему по MINSUM, это серьезный признак качества.

В проекте LKH-3 используется не как основная внутренняя реализация, а как внешний ориентир. Такой выбор оправдан: полноценный LKH-3 уже содержит большое число инженерных и алгоритмических оптимизаций, поэтому честнее сравнивать собственные версии с ним как с сильным решателем, а не пытаться представить его как часть собственной разработки. Сравнение с LKH-3 также помогает понять, где именно внутренним *lkh-wrapper* еще не хватает глубины поиска.

Ограничение LKH-3 в рамках проекта связано с интеграцией и управлением экспериментами. Внешний запуск требует подготовки файлов задачи и параметров, запуска отдельного исполняемого файла и разбора результата. Кроме того, сильный внешний решатель не всегда удобно адаптировать под исследовательские изменения, например под собственные ограничения, альтернативные метрики баланса или специальные бюджеты времени. Поэтому в работе параллельно развиваются собственные LKH-подобные версии.

Нужно добавить перед сдачей. Вставить рисунок из LKH-3_REPORT.pdf: желательно схему или фрагмент, где объясняется преобразование ограниченных задач маршрутизации к TSP/использование *penalty function*. Под рисунком нужно указать, что он иллюстрирует общий принцип LKH-3, а не внутреннюю структуру собственного кода.

2.5 Собственные LKH-подобные версии `lkh-wrapper`

Семейство `lkh-wrapper` в проекте не является прямой копией LKH-3. Это набор собственных эвристических версий, которые используют идеи LKH-подхода в упрощенном и более контролируемом виде. Главная инженерная цель этих версий --- получить валидное mTSP-решение за ограниченное время и улучшать его без полного перебора всех возможных ребер.

Ранние версии `v1--v3` использовались как переход от простых `baseline`-алгоритмов к `candidate-based` логике. Они показывают, что даже умеренное ограничение поиска перспективными соседями может дать большой выигрыш по MINSUM. Однако эти версии еще рассчитаны скорее на малые и средние инстансы.

Версии `v4--v7` были направлены на масштабирование. В них появились геометрические `candidate sets`, пространственная сетка, кэширование расстояний и регулярная проверка `time-budget`. Это важный шаг: на больших инстансах нельзя хранить и просматривать полную матрицу расстояний как основной рабочий объект. Алгоритм должен быстро получать только те расстояния и со-седства, которые реально участвуют в поиске.

Поздняя версия `lkh-wrapper-v12` ориентирована на MINSUM и быстрый возврат первого валидного решения. В ней используются отдельные бюджеты на построение начального решения и на последующее улучшение, а также идеи `cluster-aware` построения, `savings`-подхода и межмаршрутного перераспределения. По смыслу эта версия ближе всего к практическому решателю: она не просто улучшает один маршрут, а пытается управлять распределением вершин между агентами.

Сильная сторона собственных LKH-подобных версий --- прозрачность и управляемость. Их можно запускать внутри общего экспериментального контура, менять параметры, измерять влияние отдельных эвристических блоков и адаптировать под конкретную постановку. Слабая сторона --- меньшая глубина поиска по сравнению с полноценным LKH-3. Результаты экспериментов показывают, что `v12` может быть быстрее внешнего LKH-3, но обычно уступает ему по MINSUM. Это естественный компромисс между скоростью, простотой интеграции и качеством решения.

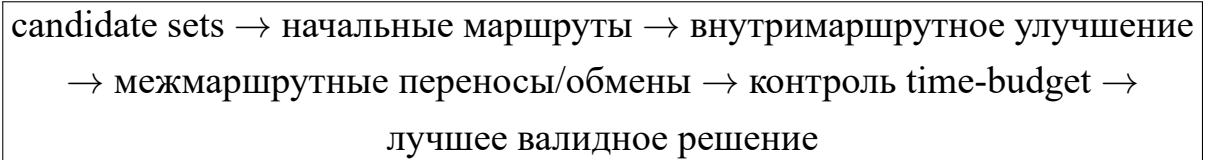


Рис. 2: Обобщенная схема LKH-подобных версий проекта

2.6 FILO2 и крупномасштабные VRP-эвристики

FILO2 рассматривается в работе как важный внешний ориентир по масштабированию, хотя сам он относится к крупномасштабным задачам маршрутизации транспорта, а не к чистой базовой постановке mTSP. Его ценность для данного проекта состоит в том, что статья Accorsi и Vigo показывает практический путь к решению очень больших маршрутизационных инстансов: алгоритм не пытается проверять все возможные перемещения, а концентрирует локальный поиск вокруг небольшой области, где изменение действительно может дать эффект [8].

В FILO2 используются несколько идей, полезных для понимания дальнейшего развития проекта. Granular neighborhoods ограничивают число рассматриваемых соседей и тем самым уменьшают стоимость локального поиска. Static move descriptors позволяют заранее описывать типы перемещений и быстрее применять их к решению. Sequential search организует улучшения последовательно, без тяжелого полного перебора всех комбинаций. Selective vertex caching хранит информацию о вершинах, которые могли быть затронуты последними изменениями, и не тратит время на области, где улучшение маловероятно.

Для mTSP эти идеи можно интерпретировать следующим образом. Если вершина была перенесена между маршрутами или внутри маршрута изменился фрагмент, нет необходимости заново анализировать весь набор клиентов. Достаточно проверить локальную окрестность измененных вершин, соседние участки маршрутов и несколько кандидатов на межмаршрутный перенос. Такой принцип хорошо согласуется с уже реализованными candidate sets и time-budget в lkh-wrapper.

При этом FILO2 нельзя без оговорок считать готовым mTSP-решателем для данной работы. В исходной статье речь идет о более общей VRP-среде, где присутствуют ограничения и цели, отличающиеся от простой MINSUM-постановки mTSP. Поэтому в отчете FILO2 используется как источник архи-

тектурных идей: как ограничивать область поиска, как кэшировать затронутые вершины и как строить эвристику, которая остается работоспособной на очень больших данных.

Нужно добавить перед сдачей. Вставить рисунок из 2306.14205v1.pdf про FILO2: лучше всего подойдет схема selective vertex caching, granular neighborhood или локальной области поиска. В подписи нужно подчеркнуть, что рисунок используется как иллюстрация идеи масштабирования VRP-эвристик, применимой к дальнейшему развитию mTSP-решателя.

2.7 ITSHA и двухэтапные mTSP-эвристики из литературы

Статья Zheng и соавторов посвящена итеративному двухэтапному эвристическому алгоритму для mTSP [10]. Для данной работы она особенно важна, потому что рассматривает именно multiple traveling salesmen problem и обсуждает как MINSUM, так и MINMAX-ориентированные варианты. Общая логика таких подходов состоит в разделении задачи на две части: сначала нужно получить разумное распределение вершин между коммивояжерами, затем улучшить порядок обхода и само распределение локальными ходами.

В ITSHA начальное решение строится с учетом кластерной структуры данных. Это близко к прикладной интуиции: если точки естественно образуют группы, то маршруты также должны использовать эту геометрию, а не распределять вершины случайно. После построения начального решения применяется вариативный локальный поиск, который меняет как порядок вершин в маршрутах, так и распределение между маршрутами. Важная идея статьи --- не ограничиваться одной простой процедурой улучшения, а чередовать несколько типов окрестностей, чтобы выходить из локальных минимумов.

По отношению к данному проекту ITSHA можно рассматривать как научное обоснование двух решений. Первое --- добавление cluster-aware начальных маршрутов в поздние версии lkh-wrapper. Второе --- необходимость межмаршрутных улучшений: только внутримаршрутного 2-opt недостаточно, если исходное назначение вершин агентам оказалось слабым. Поэтому развитие v1.2 логично вести в сторону более богатого набора relocate/swap/block moves и контроля их стоимости через candidate sets.

Нужно добавить перед сдачей. Вставить рисунок из 2201.09424v2.pdf: подойдет блок-схема ITSHA или рисунок, показывающий два этапа ``построение начального решения" и ``итеративное улучшение". После вставки нужно добавить 1--2 предложения о том, какие элементы этой схемы уже отражены в lkh-wrapper-v12.

2.8 Кластерно-генетические подходы и MMCTSP

Работа Бао и соавторов рассматривает min-max clustered traveling salesmen problem и предлагает двухэтапный генетический подход [11]. Эта постановка отличается от текущей MINSUM-версии проекта, но полезна для анализа случаев, когда входные точки имеют выраженную кластерную структуру. В таких данных важно не только построить короткие маршруты, но и решить, какие кластеры должен обслуживать каждый агент.

Двухэтапная идея выглядит естественно. На первом уровне оптимизируется порядок обхода внутри кластеров или между кластерами, на втором --- распределение кластеров между коммивояжерами. Генетический алгоритм используется для поиска хороших комбинаций, поскольку прямой перебор всех назначений быстро становится невозможным. Для данной работы это направление важно как возможное расширение: если в генераторе инстансов явно присутствуют кластеры, то алгоритм может сначала работать на уровне групп точек, а затем уточнять маршруты внутри каждой группы.

В текущем проекте полноценный генетический алгоритм не реализован, однако некоторые идеи уже близки к использованным решениям. Например, cluster-aware построение начальных маршрутов в lkh-wrapper-v12 играет похожую роль: оно пытается не разрушать естественную геометрию данных. В дальнейшем это можно усилить, добавив отдельный этап назначения кластеров агентам и сравнив его с чисто жадным построением.

Нужно добавить перед сдачей. Вставить рисунок из biomimetics-08-00238.pdf: желательно схему двухэтапного генетического алгоритма или иллюстрацию clustered TSP/mTSP. В тексте рядом нужно явно написать, что работа относится к MINMAX/clustered-варианту, поэтому используется как близкий источник идей, а не как прямое совпадение постановки.

2.9 POPMUSIC и построение candidate sets

POPMUSIC-подход Taillard и Helsgaun важен для данной работы не как отдельный mTSP-решатель, а как аргумент в пользу качественного построения candidate sets [7]. На больших TSP-инстансах полный граф слишком дорог, поэтому качество локального поиска во многом зависит от того, какие ребра попадут в набор перспективных кандидатов. Если candidate set слишком узкий, алгоритм пропустит хорошие ходы. Если он слишком широкий, время работы снова станет неприемлемым.

В статье по POPMUSIC показано, что candidate edges можно строить на основе анализа локальных подзадач и объединения информации из нескольких туров. Для проекта это подтверждает общий инженерный выбор: ускорение должно достигаться не только сокращением числа итераций, но и более умным выбором ребер, которые алгоритм вообще рассматривает. Поэтому геометрические candidate sets, сеточное разбиение пространства и кэш расстояний в `lkh-wrapper` являются не временными техническими упрощениями, а центральной частью алгоритмической идеи.

Нужно добавить перед сдачей. Вставить рисунок из POPMUSIC_REPORT.pdf или author postprint.pdf: лучше всего подойдет схема построения candidate edges или иллюстрация локальных подзадач POPMUSIC. После рисунка нужно связать его с текущей реализацией: ``в проекте используется более простая геометрическая версия этой идеи".

2.10 Сравнительная характеристика эвристик

Сводное сравнение методов показывает, что ни одна эвристика не является универсально лучшей во всех режимах. Простые методы важны для baseline и быстрой проверки. GRASP дает лучшее качество за счет многократных запусков и межмаршрутных обменов. LKH-3 задает сильный внешний ориентир качества. Собственные LKH-подобные версии занимают промежуточное место: они быстрее и лучше контролируются в экспериментальном контуре, но пока уступают полноценному LKH-3 по глубине поиска.

Метод	Основная идея	Сильная сторона	Ограничение
rand+nn	случайное разбиение и ближайший сосед внутри группы	минимальное время, простой baseline	слабое назначение вершин маршрутам
2opt+greed	жадное построение и 2-opt внутри маршрутов	быстро улучшает порядок обхода	почти не исправляет межмаршрутное распределение
grasp	RCL, много стартов, 2-opt и обмены	лучше выходит из локальных ошибок построения	дороже по времени и чувствителен к параметрам
LKH-3	локальный поиск переменной глубины, candidate sets, penalty functions	сильный внешний ориентир качества	сложнее интеграция и меньше прозрачности для модификаций
lkh-wrapper	собственные candidate-based эвристики для mTSP	управляемость, скорость, time-budget	пока меньше глубина поиска, чем у LKH-3
FILO2	гранулярные окрестности и кэш затронутых вершин	идеи масштабирования до очень больших данных	не является прямым mTSP-решателем проекта
ITSHA	двухэтапное построение и вариативный локальный поиск	хорошо соответствует mTSP и кластерной геометрии	требует более сложной реализации окрестностей
Кластерно-генетический подход	назначение кластеров и эволюционный поиск	полезен для явно кластерных данных	относится к отличающейся MINMAX/clustered-постановке

Из этого сравнения следует направление дальнейшего улучшения проекта. Базовые алгоритмы уже выполняют свою роль как контрольные ориентиры. Основной потенциал развития находится в LKH-подобной линии: нужно усилить межмаршрутные ходы, аккуратно использовать кластерную структуру входных данных и перенести из FILO2/POPMUSIC идею более умного ограничения области поиска. Тогда можно приблизиться к качеству LKH-3, сохранив при этом управляемость собственного решателя и удобство массовых экспериментов.

3 ПОСТАНОВКА ЗАДАЧИ И РЕАЛИЗАЦИЯ

3.1 Формальная постановка в проекте

Входной файл mTSP-инстанса содержит число вершин n , число агентов m и координаты всех вершин. Вершина с индексом 0 считается депо. Требуется построить m маршрутов, удовлетворяющих условиям:

1. каждый маршрут начинается в депо и заканчивается в депо;
2. каждая клиентская вершина $1, \dots, n - 1$ встречается ровно в одном маршруте;
3. значение MINSUM минимизируется или, для приближенного алгоритма, становится как можно меньше при заданном бюджете времени.

Валидность решения проверяется отдельно от вычисления целевой функции. Это принципиально важно: эвристика может вернуть численно короткие маршруты, но если какая-либо вершина пропущена или посещена дважды, такое решение нельзя учитывать в сравнении.

3.2 Экспериментальный контур

Репозиторий проекта содержит C++-реализации решателей и Python-скрипты для генерации данных, пакетного запуска и построения отчетов. Основные элементы контура:

- `run_mtsp.py` --- единая точка запуска mTSP-решателей;
- `experiments/generate_mtsp_instances.py` --- генерация синтетических инстансов;
- `experiments/run_benchmarks.py` --- пакетный запуск алгоритмов по конфигурации;
- `python/mtsp_baselines/ortools_routing.py` --- внешний baseline на OR-Tools;
- `python/mtsp_baselines/tsp_transform_lkh.py` --- baseline через построение TSP-тура и последующее разбиение на маршруты;
- `data/results/` --- CSV и JSON-артефакты экспериментов.

Генерация инстансов → запуск решателей → проверка валидности
маршрутов → пересчет MINSUM → агрегация CSV/JSON →
сравнительный анализ

Рис. 3: Схема экспериментального контура

Такое разделение уменьшает риск смещения кода алгоритма и кода оценки. Решатель отвечает за построение маршрутов, а отдельная часть проекта проверяет корректность результата и пересчитывает objective по сохраненным маршрутам.

3.3 Набор реализованных алгоритмов

Алгоритм	Роль в исследовании
rand+nn	Быстрый конструктивный baseline: случайность и ближайший сосед. Нужен как нижняя граница качества для простых решений.
2opt+greed	Жадное построение с локальным 2-opt. Позволяет оценить вклад внутримаршрутного локального поиска.
grasp	Рандомизированный многостартовый подход с restricted candidate list, 2-opt и межмаршрутными обменами.
lkh-wrapper-v1...v3	Ранние LKH-подобные версии; используются для проверки эффекта перехода от простой обертки к candidate-based логике.
lkh-wrapper-v4...v7	Версии для крупных инстансов с геометрическими кандидатами, кэшированием расстояний, бюджетами времени и ускоренными процедурами улучшения.
lkh-wrapper-v12	MINSUM-ориентированная версия: строит быстрое первое решение, использует savings/cluster-aware идеи и принимает межмаршрутные изменения по суммарной длине маршрутов.
OR-Tools GLS	Внешний эвристический ориентир на малых и средних инстансах.
TSP-transform + LKH	Внешний baseline: строит TSP-тур по клиентам и затем разбивает его на маршруты динамическим программированием.
LKH-3 baseline	Наиболее сильный внешний ориентир для части ручных сравнений.

3.4 Особенности LKH-подобной реализации

Поздние версии `lkh-wrapper` не являются прямым запуском LKH-3 внутри C++-кода. Это собственные эвристические обертки, вдохновленные LKH-подходом и адаптированные под быстрый экспериментальный контур. Их общая логика состоит из нескольких этапов:

1. построение геометрических `candidate sets`: для малых инстансов возможен точный перебор ближайших соседей, для крупных применяется пространственная сетка;
2. построение начальных маршрутов: используется жадная или кластерно-ориентированная процедура, учитывающая расстояние до депо и распределение точек;
3. локальное улучшение маршрутов: 2-opt ограничивается кандидатными ребрами, чтобы не выполнять полный квадратичный просмотр;
4. межмаршрутные изменения: вершины или блоки переносятся между маршрутами, если это уменьшает `MINSUM` или улучшает баланс;
5. соблюдение бюджета времени: крупные циклы регулярно проверяют `deadline` и возвращают лучшее найденное решение.

В `lkh-wrapper-v12` отдельно выделяется бюджет на получение первого решения и бюджет на улучшение. Такая схема важна для прикладного режима: даже если улучшение не успело завершиться, алгоритм должен вернуть валидное решение. В коде также используется кэш расстояний для крупных инстансов, так как хранение полной матрицы расстояний становится чрезмерным по памяти.

Candidate sets → fast seed routes → savings/cluster seed → route sanitizing
→ local ILS → inter-route relocate/swap → final validation

Рис. 4: Упрощенная логика работы поздней LKH-подобной версии

Нужно добавить перед сдачей. Для защиты отчета полезно добавить 2-3 изображения маршрутов: равномерный пример, кластерный пример и пример с выбросами. В репозитории уже есть JSON с маршрутами, поэтому рисунки можно построить отдельным Python-скриптом и вставить как PNG в приложение.

4 МЕТОДИКА ВЫЧИСЛИТЕЛЬНОГО ЭКСПЕРИМЕНТА

4.1 Семейства тестовых инстансов

Чтобы не проверять алгоритмы только на одном типе геометрии, в проекте используются несколько семейств синтетических данных. Такой дизайн лучше отражает разные сценарии маршрутизации и снижает риск, что алгоритм случайно хорошо работает только на равномерных точках.

Семейство	Смысл проверки	Депо
uniform	Равномерное распределение точек. Базовый сценарий без выраженной структуры.	центр
clustered-center	Несколько естественных групп клиентов. Проверяется умение алгоритма сохранять компактные группы.	центр
clustered-offset-depot	Кластерная структура при смещенном депо. Сценарий сложнее, потому что ближайшие кластеры и баланс маршрутов конфликтуют.	край
mixed-outliers / mixed	Основная масса точек находится в группах, но есть удаленные выбросы. Проверяется устойчивость к длинным хвостам.	центр
outliers	Усиленный сценарий с удаленными точками. Важен для анализа межмаршрутного перераспределения.	центр
high-m-stress	Повышенная нагрузка по числу агентов и маршрутов. Проверяется устойчивость распределения вершин между маршрутами.	центр

В ранних экспериментах использовалась сетка $n \in \{20, 50, 100, 200\}$, $m \in \{2, 3, 5\}$ и 10 повторов. Позднее она была расширена семействами инстансов и отдельными крупными задачами до $n = 100000$.

4.2 Метрики

Основная метрика качества --- значение MINSUM. Для сравнения с внешним ориентиром используется относительное отклонение:

$$\text{gap}(A, B) = 100 \cdot \frac{F(A) - F(B)}{F(B)}, \quad (4)$$

где $F(A)$ --- MINSUM исследуемого алгоритма, а $F(B)$ --- MINSUM базового или reference-решения. Если gap положителен, исследуемый алгоритм хуже ориентира по MINSUM; если отрицателен, он лучше.

Дополнительно учитываются:

- среднее время работы;
- число валидных запусков;
- число запусков, где алгоритм превзошел reference;
- баланс маршрутов, измеряемый как отношение максимальной длины маршрута к средней.

В работе важно не смешивать разные типы сравнения. OR-Tools с лимитом 5 секунд не является точным оптимумом. Это внешний эвристический ориентир. Поэтому выводы формулируются не как «алгоритм дал оптимальное решение», а как «алгоритм имеет такое-то отклонение от выбранного reference при таких-то условиях».

4.3 Валидация результатов

Каждый результат должен удовлетворять трем условиям: все маршруты начинаются и заканчиваются в депо, все клиентские вершины посещены ровно один раз, значение MINSUM пересчитано независимой функцией по сохраненным маршрутам. Такой подход особенно важен при сравнении нескольких поколений алгоритмов: ускоренная версия может выглядеть привлекательной по времени, но если она возвращает невалидные маршруты, ее нельзя учитывать в агрегатах качества.

Нужно добавить перед сдачей. В итоговом архиве к сдаче стоит приложить короткий README с командами воспроизведения: генерация инстансов, запуск benchmark, пересчет отчетов. Это усилит раздел о достоверности результатов.

5 РЕЗУЛЬТАТЫ И ИХ АНАЛИЗ

5.1 Базовое сравнение на равномерных инстансах

Первый набор экспериментов сравнивал rand+nn, 2opt+greed, grasp и lkh-wrapper-v2 на равномерных инстансах малой и средней размерности. По агрегированным результатам lkh-wrapper-v2 дал лучшее

среднее значение MINSUM среди внутренних алгоритмов, при этом оставался значительно быстрее GRASP.

Таблица 6: Агрегированное сравнение внутренних алгоритмов на равномерных инстансах

Алгоритм	Запуски	Средний MINSUM	Время, с	Валидные
rand+nn	10	1571.74	0.0005	10
2opt+greed	10	1042.94	0.0007	10
grasp	10	866.91	1.0469	10
lkh-wrapper-v2	10	821.33	0.0181	10

Результат подтверждает ожидаемую иерархию методов. Чистый randomized nearest neighbor работает почти мгновенно, но имеет худшее качество. Добавление 2-opt резко улучшает результат. GRASP дает еще более качественные маршруты, но платит за это временем. lkh-wrapper-v2 оказывается наиболее удачным компромиссом в этом наборе: его средний MINSUM ниже, чем у GRASP, а среднее время меньше примерно на два порядка.

5.2 Сравнение с OR-Tools

Для внешнего ориентира использовался OR-Tools Routing с Guided Local Search и лимитом 5 секунд. В таком сравнении все внутренние методы в среднем уступают OR-Tools по MINSUM, но разрыв существенно различается.

Таблица 7: Средний gap к OR-Tools на равномерных инстансах

Алгоритм	Gap, %	Время, с	Лучше ref.
rand+nn	114.73	0.0005	0.00
2opt+greed	47.41	0.0007	0.00
grasp	20.77	1.0469	0.00
lkh-wrapper-v2	14.48	0.0181	0.08

Эти данные не означают, что OR-Tools всегда лучше при любом временном бюджете. Его лимит в данном сравнении составляет 5 секунд, тогда как часть внутренних алгоритмов работает за миллисекунды. Однако таблица по-

лезна для оценки качества: она показывает, насколько далеко находятся быстрые эвристики от сильного внешнего решения.

5.3 Расширенный benchmark по семействам инстансов

После добавления нескольких семейств инстансов картина стала более содержательной. На расширенном наборе `lkh-wrapper-v2` и `lkh-wrapper-v3` заметно превосходят ранние версии, GRASP и простые baseline-алгоритмы по среднему MINSUM. При этом `lkh-wrapper-v3` обычно быстрее `v2`, но по качеству немного уступает или находится рядом.

Таблица 8: Средние значения по расширенному multi-family benchmark

Алгоритм	Средний MINSUM	Среднее время, с
rand+nn	1456.23	0.0007
2opt+greed	999.30	0.0008
grasp	820.59	1.5138
lkh-wrapper-v1	875.99	0.0395
lkh-wrapper-v2	677.61	0.0198
lkh-wrapper-v3	683.44	0.0149

В разрезе семейств видно, что преимущество LKH-подобных версий сохраняется не только на равномерных точках. Особенно сильный эффект наблюдается на кластерных данных: `lkh-wrapper-v2` снижает средний MINSUM относительно GRASP и ранних LKH-оберток, что согласуется с идеей использования геометрической структуры и более направленного локального поиска.

Таблица 9: Средний MINSUM по семействам для ключевых алгоритмов

Семейство	2opt+greed	grasp	lkh-v2	lkh-v3
clustered-center	811.99	663.14	536.62	539.58
clustered-offset-depot	959.50	817.75	538.31	550.34
mixed-outliers	1072.43	863.14	765.89	771.07
uniform	1153.26	938.34	869.62	872.79

5.4 Эволюция LKH-подобных версий

Сравнение `lkh-wrapper-v1`, `v2` и `v3` показывает, что переход от начальной обертки к версиям с более аккуратной candidate-based логикой дал существенный прирост качества. Средний gap к OR-Tools снизился примерно с 29.63% у `v1` до 14.48% у `v2`. Версия `v3` почти повторила качество `v2`, но работала быстрее.

Таблица 10: Сравнение ранних LKH-подобных версий с OR-Tools

Версия	Gap к OR-Tools, %	Время, с	Валидные
<code>lkh-wrapper-v1</code>	29.63	0.0297	10
<code>lkh-wrapper-v2</code>	14.48	0.0186	10
<code>lkh-wrapper-v3</code>	14.61	0.0102	10

Это важный результат для курсовой работы: он показывает не просто наличие нескольких запусков, а инженерную динамику. В проекте есть измеримый эффект от изменения алгоритма, а не только разовое сравнение готовых библиотек.

5.5 Крупные инстансы и проблема масштабирования

На крупных инстансах размером 10000--100000 вершин основная сложность смещается от качества локальных перестановок к способности алгоритма вообще успеть построить валидное решение и улучшить его в пределах бюджета. Эксперименты с версиями `v4`, `v5`, `v6` показывают, что ускорение может сопровождаться потерей качества или валидности.

Таблица 11: Версии `v4--v6` на крупных uniform/mixed инстансах с бюджетом около 30 секунд

Версия	MINSUM по валидным	Время, с	Валидность
<code>lkh-wrapper-v4</code>	$2.38 \cdot 10^8$	42.87	1.00
<code>lkh-wrapper-v5</code>	$2.63 \cdot 10^8$	30.09	1.00
<code>lkh-wrapper-v6</code>	$3.37 \cdot 10^5$	28.34	0.25

Среднее значение `v6` в этой таблице нельзя трактовать как преимущество качества, потому что валидными оказались только отдельные запуски. Более корректный вывод заключается в другом: версия `v6` показала нестабильность,

а v4 и v5 возвращали валидные решения, но v5 обычно была быстрее ценой ухудшения MINSUM. Следовательно, для итогового алгоритма требуется одновременно контролировать валидность, time-budget и качество.

5.6 Сравнение v12 с внешним LKH-3 baseline

Поздняя версия lkh-wrapper-v12 была ориентирована на MINSUM и быстрый возврат валидного решения. Ручные сравнения с lkh3-baseline показывают отчетливый компромисс: внешний LKH-3 дает более короткие маршруты, но собственная версия работает существенно быстрее и часто строит более сбалансированные маршруты.

Таблица 12: Сравнение lkh-wrapper-v12 с lkh3-baseline на n=5000, m=5

Инстанс	LKH-3	v12	v12, с	Ускорение v12
uniform	127094.46	159615.23	4.80	7.64
mixed	62667.81	75288.82	4.83	5.81
outliers	60255.93	71603.77	4.80	5.72
clustered-center	27837.74	37140.17	4.73	5.98
high-m-stress	61312.38	84752.47	4.83	5.86

В среднем на этих пяти инстансах LKH-3 лучше по MINSUM примерно на 21%, а v12 быстрее примерно в 6.2 раза. Это не подтверждает сильную версию гипотезы о превосходстве собственного LKH-подобного алгоритма над LKH-3 по качеству, но подтверждает более практичный вывод: при жестком бюджете времени собственная версия может давать валидное решение быстро, а дальнейшая работа должна быть направлена на сокращение разрыва по objective.

Таблица 13: Баланс маршрутов в сравнении LKH-3 и v12 на n=5000

Инстанс	Imbalance LKH-3	Imbalance v12
uniform	4.99	1.01
mixed	3.85	2.74
outliers	3.88	2.64
clustered-center	1.73	1.04
high-m-stress	4.99	1.11

Баланс маршрутов не является основной целью при MINSUM, однако таблица показывает интересный побочный эффект. `v12` чаще строит более равномерные маршруты, тогда как LKH-3 в режиме MINSUM может концентрировать больше нагрузки в отдельных маршрутах. Если прикладная задача требует одновременно низкий MINSUM и ограничение на максимальную длину маршрута, эту особенность можно использовать как основу для следующей версии алгоритма.

5.7 Интерпретация результатов

Выполненные эксперименты позволяют сформулировать несколько выводов.

Во-первых, простые эвристики полезны как baseline, но недостаточны для качественного решения mTSP. `rand+nn` и `2opt+greed` работают очень быстро, однако дают заметно худшую MINSUM.

Во-вторых, использование candidate-based локального поиска оправдано. Переход от ранних LKH-оберток к `v2/v3` почти вдвое снижает средний gap к OR-Tools на равномерных инстансах и дает устойчивый выигрыш на разных семействах данных.

В-третьих, масштабирование нельзя оценивать только по времени. Версия может быть быстрой, но невалидной или слишком сильно проигрывать по objective. Поэтому в отчете отдельно фиксируются valid runs и не делается вывод о качестве по строкам, где валидность низкая.

В-четвертых, внешний LKH-3 пока остается более сильным ориентиром по качеству. Собственная `v12` версия выигрывает по скорости и балансу, но проигрывает по MINSUM. Это честный и важный результат: он показывает, что проект находится не в состоянии «готовый промышленный решатель лучше всех», а в состоянии исследовательского прототипа с понятным направлением развития.

Нужно добавить перед сдачей. Если останется время, стоит добавить парный статистический тест или хотя бы таблицу wins/losses/ties по одинаковым инстансам для пар `lkh-wrapper-v2` vs `grasp`, `v12` vs `lkh3-baseline`. В материалах уже есть per-instance CSV/JSON, поэтому это можно сделать без новых запусков.

6 ДОСТОВЕРНОСТЬ, ОГРАНИЧЕНИЯ И РИСКИ

Достоверность результатов обеспечивается несколькими мерами. Все алгоритмы запускаются через единый runner, что снижает риск различий в формате входа и выхода. Для каждого решения проводится независимая проверка посещения вершин и пересчет MINSUM. Результаты сохраняются в CSV/JSON, что позволяет повторно агрегировать данные без ручного переноса чисел. Сравнения выполняются на одних и тех же инстансах, поэтому различия между алгоритмами связаны с их поведением, а не с различием входных данных.

При этом работа имеет ограничения. Во-первых, часть экспериментов выполнена на синтетических данных. Это правильно для контролируемого исследования, но перед прикладным использованием алгоритмы нужно проверить на реальных географических наборах. Во-вторых, не для всех крупных запусков известна полная аппаратная конфигурация. В-третьих, некоторые сравнения имеют разный временной бюджет: OR-Tools запускался с лимитом 5 секунд, а собственные алгоритмы в ранних экспериментах часто работали значительно меньше. Поэтому выводы о «лучшем» алгоритме делаются отдельно для качества и отдельно для времени.

Еще одно ограничение связано с целью MINSUM. Она минимизирует суммарную длину маршрутов, но не гарантирует справедливого распределения нагрузки между агентами. В результатах видно, что LKH-3 может давать лучший MINSUM, но менее сбалансированные маршруты. Если практическая задача требует ограничить максимальную длину маршрута, постановку нужно расширить до MINMAX или многокритериальной цели.

Основной технический риск проекта --- рост времени локального поиска на больших инстансах. Для его снижения используются candidate sets, пространственная сетка, кэш расстояний и проверка бюджета времени. Второй риск --- невалидные решения в ускоренных версиях. Он снижается отдельной фазой sanitize/complete routes и независимой валидацией.

ЗАКЛЮЧЕНИЕ

В ходе работы была рассмотрена задача множественного коммивояжера с одним депо и целевой функцией MINSUM. Были изучены классические и современные подходы к TSP/mTSP, включая локальный поиск, GRASP, LKH, LKH-3, candidate sets и идеи масштабирования для крупных маршрутизацион-

ных задач. На основе этого был реализован экспериментальный контур, позволяющий генерировать инстансы, запускать несколько классов алгоритмов, проверять валидность маршрутов и агрегировать результаты.

Эксперименты показали, что LKH-подобные методы существенно превосходят простые baseline-алгоритмы по качеству решения. На равномерных инстансах `lkh-wrapper-v2` получил средний gap к OR-Tools около 14.48%, тогда как GRASP имел около 20.77%, `2opt+greedy` --- 47.41%, а `rand+nn` --- 114.73%. На расширенном наборе семейств `lkh-wrapper-v2` и `v3` также дали лучшие средние значения MINSUM среди внутренних алгоритмов.

При переходе к крупным задачам была выявлена ключевая инженерная проблема: ускорение алгоритма должно контролироваться вместе с валидностью и качеством. Версии `v4/v5/v6` показали разные стороны этого компромисса: `v5` быстрее `v4`, но обычно хуже по MINSUM, а `v6` не обеспечила достаточную валидность. Поздняя MINSUM-ориентированная версия `v12` в ручных сравнениях с LKH-3 оказалась быстрее примерно в 6 раз на инстансах $n = 5000$, но уступила по суммарной длине маршрутов примерно на 21%. При этом `v12` часто давала более сбалансированные маршруты, что может быть полезно для дальнейшей многокритериальной постановки.

Таким образом, цель работы в исследовательском смысле достигнута: разработан и проанализирован набор эвристических методов для mTSP, построен воспроизводимый экспериментальный контур, получены сравнения с базовыми и внешними алгоритмами, выявлены сильные и слабые стороны текущих решений. Рабочая гипотеза подтверждена частично. LKH-подобные эвристики действительно лучше простых конструктивных методов и GRASP на рассмотренных наборах, но пока не превосходят внешний LKH-3 по качеству MINSUM. Дальнейшее развитие проекта целесообразно направить на совмещение скорости `v12` с более сильной процедурой улучшения, добавление статистического анализа и проверку на реальных маршрутизационных данных.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Lin S., Kernighan B. W. An Effective Heuristic Algorithm for the Traveling-Salesman Problem // Operations Research. 1973. Vol. 21, No. 2. P. 498--516.
2. Helsgaun K. An Effective Implementation of the Lin--Kernighan Traveling Salesman Heuristic // European Journal of Operational Research. 2000. Vol. 126, No. 1. P. 106--130.
3. Helsgaun K. An Extension of the Lin--Kernighan--Helsgaun TSP Solver for Constrained Traveling Salesman and Vehicle Routing Problems. Roskilde University, 2017.
4. Bektaş T. The Multiple Traveling Salesman Problem: An Overview of Formulations and Solution Procedures // Omega. 2006. Vol. 34, No. 3. P. 209--219.
5. Jonker R., Volgenant T. Technical Note --- An Improved Transformation of the Symmetric Multiple Traveling Salesman Problem // Operations Research. 1988. Vol. 36, No. 1. P. 163--167.
6. Croes G. A. A Method for Solving Traveling-Salesman Problems // Operations Research. 1958. Vol. 6, No. 6. P. 791--812.
7. Taillard É. D., Helsgaun K. POPMUSIC for the Travelling Salesman Problem // European Journal of Operational Research. 2019. Vol. 272, No. 2. P. 420--429.
8. Accorsi L., Vigo D. Routing One Million Customers in a Handful of Minutes. 2023. arXiv:2306.14205.
9. Feo T. A., Resende M. G. C. Greedy Randomized Adaptive Search Procedures // Journal of Global Optimization. 1995. Vol. 6, No. 2. P. 109--133.
10. Zheng J., Hong Y., Xu W., Li W., Chen Y. An Effective Iterated Two-stage Heuristic Algorithm for the Multiple Traveling Salesmen Problem. 2022. arXiv:2201.09424.
11. Bao X., Wang G., Xu L., Wang Z. Solving the Min-Max Clustered Traveling Salesmen Problem Based on Genetic Algorithm // Biomimetics. 2023. Vol. 8, No. 238.
12. Russell R. A. An Effective Heuristic for the M-Tour Traveling Salesman Problem with Some Side Conditions // Operations Research. 1977. Vol. 25, No. 3. P. 517--524.
13. Reinelt G. TSPLIB --- A Traveling Salesman Problem Library // ORSA Journal

- on Computing. 1991. Vol. 3, No. 4. P. 376--384.
14. Johnson D. S. A Theoretician's Guide to the Experimental Analysis of Algorithms // Data Structures, Near Neighbor Searches, and Methodology. 1999. P. 215--250.
 15. Hooker J. N. Testing Heuristics: We Have It All Wrong // Journal of Heuristics. 1995. Vol. 1, No. 1. P. 33--42.
 16. Google OR-Tools. Routing Library Documentation. Электронный ресурс: <https://developers.google.com/optimization/routing>.
 17. Dantzig G. B., Ramser J. H. The Truck Dispatching Problem // Management Science. 1959. Vol. 6, No. 1. P. 80--91.
 18. Toth P., Vigo D. Vehicle Routing: Problems, Methods, and Applications. SIAM, 2014.
 19. Материалы репозитория `mtsp-routing-optimization`: исходный код решателей, скрипты экспериментов и CSV/JSON-артефакты результатов. Локальный архив проекта, 2026.

ПРИЛОЖЕНИЕ А. ЧТО НУЖНО ДОБАВИТЬ В ФИНАЛЬНУЮ ВЕРСИЮ ПЕРЕД СДАЧЕЙ

В основной текст уже включена структура полноценного итогового отчета: реферат, определения, введение, обзор предметной области, постановка задачи, реализация, методика эксперимента, результаты, ограничения, заключение и список источников. Ниже перечислены элементы, которые желательно добавить после уточнения данных.

1. Точный объем отчета: число страниц, таблиц, рисунков, источников и приложений.
2. Полная аппаратная конфигурация экспериментов: CPU, RAM, ОС, компилятор, режим сборки, число потоков.
3. Скриншоты или PNG-визуализации маршрутов для 2--3 характерных инстансов.
4. Рисунки из статей в папке `docs/materials`: схема LKH-3, схема FILO2, блок-схема ITSNA, иллюстрация clustered/генетического подхода и рисунок по POPMUSIC/candidate sets. В главе 2 уже отмечены места, куда эти изображения логично вставить.
5. Таблица `wins/losses/ties` по одинаковым инстансам для ключевых пар алгоритмов.
6. Полная таблица `per-instance` результатов в приложении или отдельном CSV-архиве.
7. Команды воспроизведения экспериментов и ссылка на репозиторий/архив исходного кода.
8. Финальная сверка всех чисел после последнего запуска `benchmark`, если будут добавлены новые эксперименты.

ПРИЛОЖЕНИЕ Б. РЕКОМЕНДУЕМЫЙ СОСТАВ ПОЛНОЙ ТАБЛИЦЫ ЭКСПЕРИМЕНТОВ

Полную таблицу не следует перегружать в основной части отчета. Ее лучше вынести в приложение или приложить отдельным CSV. Рекомендуемый набор столбцов:

Столбец	Содержание
instance_family	Семейство инстанса: uniform, clustered-center, mixed, outliers и т.д.
instance	Имя конкретного файла с тестом.
node_count	Число вершин.
salesman_count	Число агентов.
solver	Название алгоритма.
valid	Результат проверки корректности маршрутов.
objective	Пересчитанное значение MINSUM.
time_seconds	Время выполнения.
reference_objective	Значение внешнего ориентира, если он использовался.
relative_gap_percent	Относительное отклонение от reference.
imbalance	Отношение максимальной длины маршрута к средней длине маршрута.
seed	Seed запуска, если алгоритм стохастический.